Development Plan
FitFolio

Team #1, Burn n' Earn
Davis Lenover
Preston Grewal
Angad Chhabra
Farid Bastoros
Zaid Alsaadi

Table 1: Revision History

Date	**Developer(s)**	**Change**
Sep 22–27, 2026	Davis Lenover	Initial revision of 5 Team Communication Plan, 10 Expected Technology, and the Team Charter's Accountability and Teamwork.
Sep 23–27, 2026	Farid Bastoros	Initial revision of 7 Workflow Plan, 8 Project Decomposition and Scheduling, 9 Proof of Concept Demonstration Plan, and the Team Charter's Team Building and Decision Making.
Sep 26–27, 2026	Preston Grewal	Initial revision of 1 Confidential Information, 2 IP to Protect, 3 Copyright License, and 4 Team Meeting Plan.
Sep 26–27, 2026	Angad Chhabra	Initial revision of 6 Team Member Roles, 12 Coding Standard, and the Team Charter's External Goals and Attendance.
Sep 26–27, 2026	Zaid Alsaadi	Initial revision of 11 Use of AI and LLMs and 13 Generative AI Use Disclosure.
Sep 27, 2026	Farid Bastoros	Reflection answers for Questions 1-3.
Sep 27–28, 2026	Farid Bastoros	Full review of the document against the rubric and checklist, identifying inconsistencies and formatting issues for revision.

This document describes how the team will develop FitFolio. It records the team's decisions on meetings, communication, roles, and Git workflow, breaks the project into scheduled tasks, and outlines the proof of concept demonstration, the expected technologies, and the coding standard. The appendices contain the team's disclosure of generative AI use, reflections, and team charter.

1 Confidential Information

There is no confidential industry information to protect.

2 IP to Protect

There is no intellectual property (IP) to protect.

3 Copyright License

Our team is using the MIT License, which can be found in the LICENSE file of the repository.

4 Team Meeting Plan

The team will meet every Monday after the lecture at 6:30 PM. If there is nothing to discuss in a given week, such as right after a deliverable is submitted, the meeting chair may cancel that week's meeting by notifying the team on Microsoft Teams at least 24 hours in advance. Members are encouraged to meet in person on campus, but those who cannot attend can join through Microsoft Teams.

Meetings with the supervisor, Chris Schankula, are arranged as needed with at least two days' notice, through the team liaison. They will be held in person when possible, and on Microsoft Teams otherwise.

Meetings follow an agenda that lists questions and topics for discussion. The meeting chair posts the agenda as a GitHub issue at least 24 hours before the meeting, and the minutes are recorded in that issue.

5 Team Communication Plan

This section outlines how the team is expected to communicate both internally and externally.

5.1 General Intra-team Communication

This subsection outlines how internal team communication shall be handled.

- **Microsoft Teams** is used for day-to-day discussion, scheduling, and virtual meetings.
- **GitHub Issues** are used for all development tasks, bugs, and feature discussion. Every meeting (team, supervisor, TA) has an issue created from the corresponding template, with the agenda posted beforehand and minutes added afterwards.
- **Pull requests** are used for all code and document review. Discussion about a change stays on its pull request unless otherwise noted.
- Any decision that affects the project must be recorded in the relevant GitHub issue or meeting minutes. A decision is final once it is recorded there. Decisions that exist only in Teams chat are *not* considered final.
- Team members are expected to acknowledge messages that mention/concern them within 12 hours (24 hours on weekends). It is expected that team members are proactive in reading any team communication regardless of platform.
- Pull request reviews shall be completed within 48 hours of the request, or within 24 hours during the final three days before a deadline (see Section 7.3).

5.2 External Communication

This subsection outlines external communication between the team and outside sources.

- Communication between the supervisor (Chris Schankula) and the team will be mediated by Davis. It is their responsibility to forward information to/from the supervisor within 12 hours.
- Should Davis be unavailable to handle supervisor communication for whatever reason, they must inform the group with as much advance notice as possible. A temporary liaison will be appointed by the entire group along with communication to the supervisor that states this change.

6 Team Member Roles

The team will use adaptive roles over the course of the project. Instead of assigning each member to one responsibility for the entire project, roles can

change between deliverables and development phases, depending on the needs of the team and its members. Table 2 lists the initial assignments.

Table 2: Initial role assignments

Role	**Initial Member(s)**
Team Liaison	Davis
Meeting Chair	Farid
Notetaker	Zaid
QA Reviewer (see Quality Assurance)	Farid, Angad
Health Data Integration	Preston, Zaid
Market Engine and Financial Data	Davis, Farid
Front-End Developer	Angad, Zaid
Back-End Developer	Farid, Davis

The responsibilities of each role are as follows:

- **Team Liaison:** Acts as the primary point of communication between the team, the supervisor, and course staff.
- **Meeting Chair:** Posts each meeting's agenda, leads team meetings, keeps discussion focused, and makes sure action items are assigned.
- **Notetaker:** Records decisions, deadlines, and questions in the meeting's GitHub issue, as well as information discussed during lectures.
- **QA Reviewer:** Checks the full deliverable against its checklist and rubric before submission, as described in the Team Charter.
- **Health Data Integration:** Integrates smartphone health platform data, processes fitness activity, and connects completed activity to the reward system.
- **Market Engine and Financial Data:** Handles virtual portfolio calculations, transaction processing, market data integration, and portfolio history.
- **Front-End Developer:** Builds the mobile application's user interface and the presentation of fitness and portfolio information.
- **Back-End Developer:** Builds server-side application logic, APIs, authentication, and communication between the client and external services.

Roles rotate as follows. The Meeting Chair, Notetaker, and QA Reviewer rotate at the start of each deliverable, so every member takes on each role during the year. Technical roles stay with the same members for continuity, but each area has at least two members, so no part of FitFolio depends on a single person. A member can ask to change technical roles at any weekly meeting, and

the change is decided using the team's decision-making process. All members contribute to development, testing, documentation, and code review regardless of their assigned roles.

7 Workflow Plan

This section describes how the team will use Git and GitHub to collaborate on both source code and documentation, how work will be tracked through issues, and how Continuous Integration and Continuous Deployment (CI/CD) will keep the project stable. It is written to serve as a guide for current and future contributors. The repository is available at BurnNEarn GitHub Repository.

7.1 Branches

The repository will have exactly one permanent branch:

- `main`: the protected, stable branch representing the latest reviewed state of the project. Direct pushes are not permitted. All changes must arrive through a pull request that has been approved by at least one other team member and has passed all CI checks.

All other branches are short-lived and must follow the naming convention `<type>/<name>-<description>`, where `type` is one of the categories in Table 3, `name` is the author's first name in lowercase, and `description` is a brief summary of the work in two to four words, separated by hyphens.

Table 3: Branch Types and Naming Examples

Type	Used For	Example
`docs`	Documentation sections	`docs/farid-dev-plan-workflow`
`feature`	New app functionality	`feature/davis-step-sync`
`fix`	Bug fixes	`fix/angad-portfolio-rounding`
`chore`	CI, configuration, and repository maintenance	`chore/preston-latex-build`

Branches are not shared between members. Each member creates their own branch for each unit of work. For documentation, each member creates one branch per deliverable covering the sections assigned to them. For code, each feature or fix gets its own branch. Once a branch's pull request is merged into `main`, the branch is deleted. Before opening a pull request, the author updates their branch with the latest changes from `main` and resolves any merge conflicts locally, so that conflicts are never resolved during review.

7.2 Commits

Commits should be small and focused on a single change, rather than combining unrelated edits. Commit messages should begin with a short summary in the imperative mood (e.g., "Add step count permission request") and reference the related issue number where applicable (e.g., `#14`), which links the commit to that issue on GitHub. Issues are closed through the pull request rather than individual commits, as described in Section 7.3.

7.3 Pull Requests

When the work on a branch is complete, the author follows these steps:

1. Open a pull request into `main` with a title summarizing the change and a description of what was changed and why. The description must link the related issue using a closing keyword (e.g., `Closes #14`), which automatically closes the issue when the pull request is merged.
2. Request at least one reviewer. The reviewer should be a member working in a related area, or the whole team if the change affects everyone, such as shared document sections or the overall architecture.
3. Address the reviewer's comments, which are left directly on the relevant lines of code or documentation.
4. Once the pull request has one approval and all CI checks pass, the author merges it using a *merge commit*, which preserves every individual commit from the branch on `main`. This keeps a complete record of each member's contributions throughout the project. The author then deletes the branch.

Reviewers are expected to respond to a review request within 48 hours, or within 24 hours during the final three days before a deadline. For documentation, reviewers also check the changes against the deliverable's checklist and rubric before approving.

7.4 Tags

Deliverables are marked directly from the `main` branch, so a Git tag will be created on `main` at each deliverable's deadline to record exactly what was marked. Document deadlines use the format `<document>-<revision>` (e.g., `dev-plan-rev0`, `srs-rev0`, `design-rev-1`), and demonstrations use the format `<demo>-demo` (e.g., `poc-demo`, `rev0-demo`, `final-demo`). The team member who merges the final pull request before a deadline is responsible for creating its tag. Tags allow the team to return to the exact state of the project at each deadline, and to compare revisions when addressing feedback.

7.5 Issue Management

GitHub Issues will serve as the team's ticketing system, so that all work is visible, assigned, and traceable.

- **Issue size:** Each issue should represent work that can be completed in a single sitting of roughly two to three hours. Larger tasks are split into multiple issues. For documentation, every section of each document has its own issue, titled with its deliverable and section (e.g., `[Development Plan] 7 Workflow Plan`).
- **Ownership:** Every task issue is assigned to one owner, who is responsible for completing it and moving it through the project board described in Section 8.1.
- **Milestones:** Each deliverable is a milestone whose due date matches the deadline in Table 7. Every issue related to a deliverable is added to its milestone, so the team can see how much work remains before each deadline.
- **Meetings and lectures:** An issue is created for every team meeting, supervisor meeting, and lecture, and is tracked in its own column on the project board. At least 24 hours before a meeting, the agenda and any questions are added to the issue. Afterwards, notes and action items are recorded, each action item becomes its own issue, and the meeting issue is closed. Lecture issues record the topic, date and time, and attendance.

The repository includes the issue templates listed in Table 4, so that every issue contains consistent information.

Table 4: Issue Templates

Template	**Contents**
Documentation Task	Deliverable, section, and the checklist items the section must satisfy
Feature	Description of the functionality and acceptance criteria
Bug Report	Steps to reproduce, expected and actual behaviour, and device and operating system
Meeting	Meeting type, date and time, attendance, agenda, questions, notes, and action items
Lecture	Topic, date and time, and attendance

Issues are classified using the labels in Table 5. Every issue that belongs to a deliverable receives that deliverable's label, and every issue receives one type label and, where relevant, one area label. Together, these allow the team to filter issues by deliverable, by kind of work, or by part of the app.

Table 5: Issue Labels

Category	Label	Meaning
Deliverable	`Problem Statement and Goals`	Problem Statement and Goals document
	`Development Plan`	Development Plan and Proof of Concept Plan
	`Requirements`	Software Requirements Specification
	`Hazard Analysis`	Hazard Analysis document
	`V&V Plan`	Verification and Validation Plan
	`Design Document`	Design Document (all revisions)
	`V&V Report`	Verification and Validation Report
	`Demo`	Proof of Concept, Revision 0, Final, and EXPO demonstrations
Type	`documentation`	Writing or revising a document section
	`reflection`	Reflection components of a deliverable
	`feature`	New functionality
	`enhancement`	Improvement to existing functionality
	`bug`	A defect that must be fixed
	`meeting`	Team or supervisor meeting
	`lecture`	Course lecture
Area	`frontend`	User interface of the mobile app
	`backend`	Server, database, and accounts
	`health-data`	Fitness tracking and health platform integration
	`market-engine`	Asset prices, portfolios, and market scenarios
	`social`	Social feed and sharing

7.6 Continuous Integration and Continuous Deployment

GitHub Actions will run automated checks on the repository, so that errors are caught when a change is proposed rather than after it reaches `main`:

- **Documentation:** On every pull request that changes the `docs` folder, all LaTeX documents are compiled. The check fails if any document does not build, and the generated PDFs are saved as downloadable artifacts of the workflow run, linked from the pull request, so reviewers can read the rendered changes.
- **Code:** Once development begins, every pull request and every push to `main` runs a linter and the automated test suite in the `test` folder. Branch protection prevents a pull request from being merged if any check fails.
- **Deployment:** Once the core features are stable, targeted for Revision 0, every merge into `main` will automatically produce an internal test build of

the mobile app, so the team and supervisor can always install the latest version on their phones.

If a check fails, the pull request's author is responsible for fixing it before requesting another review.

8 Project Decomposition and Scheduling

This section describes how the team will use GitHub Projects to track work, how each deliverable will be broken down into manageable tasks, and how the project will be scheduled around both course deadlines and earlier team deadlines.

8.1 GitHub Projects

The team will use a GitHub Project board, *BurnNEarn-TrackingBoard*, as the central place to track all issues. It is available at FitFolio GitHub Project. Every task issue moves through the first four columns in Table 6 in order, while meeting and lecture issues are kept in their own columns so they do not clutter the task workflow.

Table 6: Project Tracking Board Columns

Column	**Description**
Backlog	Tasks that have been identified but not yet started. New issues are added here automatically.
In Progress	Tasks being actively worked on. The owner moves an issue here when they create its branch.
In Review	Tasks with an open pull request awaiting review. The owner moves an issue here when they open the pull request.
Done	Tasks whose pull request has been approved and merged. Issues move here automatically when they are closed.
Meetings	Team and supervisor meetings, with their agendas, notes, and action items.
Lectures	Course lectures, with their topic, date and time, and attendance.

The board can be filtered by milestone, assignee, or label. Filtering by the current milestone shows how much work remains before the next deadline, and filtering by assignee shows each member's workload, which helps the team balance tasks during weekly meetings.

8.2 Task Decomposition

At the start of each deliverable, the team breaks the work into issues and adds them to that deliverable's milestone. Work is decomposed differently depending on its type:

- **Documentation** is broken down by section, with one issue per section titled with its deliverable and section number (e.g., `[Development Plan] 8 Project Decomposition and Scheduling`). Each section can then be written, reviewed, and merged independently.
- **Software** is broken down by feature, and each feature is split further into issues that fit within a single sitting and can be tested on their own.

Issues are assigned during the weekly team meeting, so that every member knows who is responsible for what. Assignments balance two goals: using each member's strengths to maintain quality, and giving every member experience across different parts of the project, so that no part of FitFolio depends on a single person. When a member takes on a task in an unfamiliar area, they are paired with a member who has experience in that area and can offer guidance.

8.3 Schedule

The project is scheduled around the course deliverables, each of which is a GitHub milestone. For every deliverable, the team sets a team deadline three days before the course deadline. All work must be merged into `main` by the team deadline, and the remaining three days are reserved for peer review, a quality assurance check of the full deliverable against its checklist and rubric, and final edits. For demonstrations, the team deadline is a full rehearsal of the demonstration, held three days before the first demonstration day. Table 7 lists both deadlines for every deliverable. Unless otherwise stated, all course deadlines are at 11:59 p.m.

Table 7: Team and Course Deadlines

Deliverable	Team Deadline	Course Deadline
Problem Statement, POC Plan, Development Plan	Sep 25, 2026	Sep 28, 2026
Requirements Document and Hazard Analysis Revision 0	Oct 16, 2026	Oct 19, 2026
V&V Plan Revision 0	Oct 30, 2026	Nov 2, 2026
Design Document Revision −1 and CD-Fall Revision 0	Nov 13, 2026	Nov 16, 2026
Proof of Concept Demonstration	Nov 20, 2026	Nov 23 - Dec 4, 2026
Design Document Revision 0	Jan 15, 2027	Jan 18, 2027
Revision 0 Demonstration	Jan 29, 2027	Feb 1 - Feb 12, 2027
V&V Report Revision 0 and CD-Winter Revision 0	Mar 5, 2027	Mar 8, 2027
Final Demonstration (Revision 1)	Mar 12, 2027	Mar 15 - Mar 25, 2027
EXPO Demonstration (poster due)	Mar 28, 2027	Mar 31, 2027
Final Video	Mar 28, 2027	Mar 31, 2027
Final Documentation (Revision 1)	Mar 28, 2027	Mar 31, 2027

To keep work on track between deadlines, each member reports progress and any blockers on their assigned issues during the weekly team meeting. A member who expects to miss a team deadline must tell the team on Microsoft Teams at least 24 hours beforehand, so that the task can be supported or reassigned. Feedback on each Revision 0 deliverable will be recorded as issues and addressed throughout the year, rather than left until the Final Documentation.

9 Proof of Concept Demonstration Plan

This section identifies the main risks to the success of FitFolio, describes what the proof of concept (PoC) demonstration will show to convince the team that these risks can be overcome, and outlines what the team will do if the demonstration shows that a risk cannot be overcome as planned.

9.1 Main Risks

FitFolio depends on two external systems that no team member has worked with before. Both are implementation risks: if either cannot be made to work, the app's core loop of earning currency through activity and investing it in real assets cannot be built.

1. **Reading fitness data from the phone's health platform.** FitFolio's core promise is that activity is tracked automatically. This requires reading step data through Google Health Connect on Android and Apple HealthKit on iOS, which each have their own Application Programming Interfaces (APIs), permission models, and restrictions on when data can be read. Since FitFolio will be built with React Native, it cannot access these platforms directly, and will depend on third-party libraries to bridge between the app and each platform. These libraries may be incomplete, poorly maintained, or out of date with the latest platform versions, which the team does not control. If step data cannot be read reliably, the app cannot award currency fairly.

2. **Valuing portfolios with real market prices.** The virtual portfolio must follow real asset prices, but market data providers limit how many requests can be made, especially on free plans. If prices cannot be retrieved often enough within these limits, portfolio values will not follow the market closely enough to teach users how investing works.

9.2 Demonstration

The PoC will show one minimal path through FitFolio's core loop, running on a physical Android phone:

1. **Read the day's step count** (Risk 1). After the user grants permission, the app reads the current day's steps from Google Health Connect and displays them. This succeeds if the count matches the phone's own health app, and updates after the team member walks and reopens the app.

2. **Convert steps into virtual currency** (connects Risks 1 and 2). The step count is converted into currency using a simple fixed rule, and the balance is displayed.

3. **Buy a real asset with virtual currency** (Risk 2). The user spends their balance on shares of one real asset, such as a stock or Exchange-Traded Fund (ETF), at its current market price.

4. **Show the portfolio's value following the market** (Risk 2). The portfolio's value is recalculated each time a new price is retrieved. This succeeds if the displayed value matches the asset's public market price at each update, and the number of requests used shows that the chosen update interval fits within the provider's free limits.

If time allows, the same steps will also be shown on an iPhone using Apple HealthKit. This is a stretch goal rather than a requirement, so that the core demonstration stays achievable.

The PoC is deliberately not a prototype. It will have a minimal user interface and will not include user accounts, workouts, the social feed, or market scenarios. Its only purpose is to show that the two main risks can be overcome. The team

will rehearse the full demonstration by the team deadline in Table 7, and the demonstrated version will be tagged `poc-demo` on `main`.

9.3 If the PoC Fails

If the demonstration shows that a risk cannot be overcome as planned, the team will adjust the project's scope rather than its core idea:

- **If step data cannot be read reliably:** The team would first limit syncing to when the user opens the app, rather than in the background, which avoids most platform restrictions. If a third-party library proves unreliable, the team would switch to an alternative library, or write a small amount of platform-specific code to access the health platform directly. If one platform still cannot be supported, FitFolio would support only the platform that works, and the team would discuss with the supervisor whether the second platform should become a future goal. If neither platform works, users would enter their activity manually as a last resort. Since this weakens FitFolio's promise of automatic tracking, the team would discuss this change in scope with the supervisor before adopting it.
- **If market data limits are too restrictive:** The team would reduce how often prices are updated (e.g., every 15 minutes, or once per day), have a server retrieve each price once and share it across all users rather than requesting it per user, and limit the app to a curated list of assets. If a provider's limits or cost are still unworkable, the team would switch to a different provider.

9.4 Secondary Risk

A secondary risk is that the market scenarios, such as replays of past downturns, depend on historical price data that may be limited or costly to access. Since this feature is not part of the core loop, it will not be included in the PoC. If historical data proves difficult to obtain, the team would reduce the number of scenarios offered, or build them from a small set of stored historical datasets.

10 Expected Technology

The following section details the expected technologies for the systems the team will be developing.

10.1 Operating Systems

The primary target operating systems for this app will be Android 17+ and iOS 27+. This is due to the nature of the testing equipment available for all group members and the supervisor.

10.2 Languages

It is expected that React Native (TypeScript) will be used for the front-end. This will allow the team to write code that works on both platforms at once. The backend will be written in Python because its libraries cover the backend's main tasks: requesting stock data from web APIs, serving the app's API (see Section 10.3), and modeling price data within the market simulation engine using NumPy and pandas.

Alternatives considered: Flutter was considered for the front-end, but it uses Dart, a language less familiar to the team than TypeScript. TypeScript (Node.js) was considered for the backend so the whole codebase would share one language, but Python's numerical libraries (NumPy and pandas) are better suited to the market simulation engine.

10.3 Backend Framework

The app will communicate with the Python backend through a REST API over HTTPS, exchanging data as JSON. The backend will be built with FastAPI, which validates requests using Python type hints and automatically generates API documentation, helping keep the front-end and backend in agreement on what data is sent. Its support for asynchronous requests also suits fetching market data from external APIs.

Alternative considered: Django (with Django REST Framework) was considered, but it includes many features the project does not need (e.g., server-rendered pages), which adds setup and boilerplate.

10.4 Market Data

Real-world stock market data will be retrieved from the Finnhub API, whose free tier provides current stock prices and company information. To stay within its request limits, fetched data will be cached in the database (see Section 10.6).

Alternative considered: yfinance, an unofficial library that reads Yahoo Finance data, was considered as it is free and simple to use. However, it is not an official API and can stop working without notice whenever Yahoo changes its website.

10.5 Libraries

The following libraries are expected to be used for React Native:

- @kingstinct/react-native-healthkit - Provides access to HealthKit on iOS.
- react-native-health-connect - Provides access to Health Connect on Android.

The following libraries are expected to be used for Python:

- NumPy - Handles numerical computing quickly (e.g., vector operations).

- pandas - Data manipulation and analytics.

Alternative considered: Writing custom native modules (in Swift and Kotlin) to access HealthKit and Health Connect directly was considered, but existing bridge libraries avoid the need to write and maintain code in two additional languages.

10.6 Databases

User data, portfolios, the social feed and cached market data (as pulling constantly from an API will likely be expensive) will be stored in PostgreSQL.

Alternative considered: MongoDB was considered for its flexible document storage. However, the app's data is highly relational (e.g., users, their portfolios, trades and friends), and trades must be recorded reliably, which PostgreSQL's relations and transactions handle well.

10.7 Testing

For unit testing, Jest will be used for the front-end as this works directly with React Native. For the backend, pytest will be used as this works well with Python and doesn't require a lot of boilerplate class code (unlike Python's built-in unittest), therefore allowing the team to test much quicker.

10.8 Linting and Formatting

ESLint and Prettier will be used for the front-end: ESLint catches common mistakes in TypeScript code, and Prettier formats code automatically. Prettier will be configured to use four spaces for indentation, matching the Coding Standard (see Section 12). For the backend, Ruff will be used as a single tool that both lints and formats Python code. These checks will run in CI on every pull request and must pass before it can be merged (see Definition of Done).

Alternative considered: For Python, Black (formatting) with Flake8 (linting) was considered, but Ruff covers both in one tool and runs considerably faster.

10.9 Continuous Integration

GitHub Actions will be used for continuous integration, as described in Section 7.6. For backend tests, CI will also run a PostgreSQL container so that features that query the database can be tested.

10.10 Tools

Git and GitHub will be used for version control, and GitHub Projects for tracking work, as described in Sections 7 and 8.

11 Use of AI and LLMs

The team plans to use Artificial Intelligence (AI) and Large Language Models (LLMs) to help with both code and documentation development throughout the FitFolio project. When working on code we may use them to explain unfamiliar libraries, compare possible approaches, understand error messages, and suggest fixes or test cases. Team members will review these suggestions and decide whether they fit the project. For the documentation of this project, we may use LLMs to brainstorm ideas, organize sections, improve unclear wording, and identify missing information or contradictions. The final content will reflect our team's decisions and the work we have actually completed.

We plan to use ChatGPT and Claude for these tasks. Team members may use either tool depending on what they need help with. For example, someone might use ChatGPT to understand an error message or Claude to review a section of code. We will record the tools and model versions used where available along with any AI assistance that meaningfully contributes to each deliverable.

Before including AI-generated code, a team member will read it and understand how it works, and then check that it meets the intended requirements for the project. We will run relevant tests that include normal cases and situations where inputs are missing or invalid. For example, code that calculates fitness rewards or virtual portfolio values will be checked using inputs with known expected results. We will also check claims about libraries and services against their documentation. For written content, we will check factual claims and references against the original sources and make sure the text agrees with our project scope and team decisions. Changes will go through teammate review in a pull request, with corresponding justification for including the generated content, before being merged. We will correct or remove any suggestions that we cannot verify.

12 Coding Standard

The development team will adopt a standard based on the Microsoft TypeScript Coding Guidelines as the primary guideline for TypeScript code written throughout the project. The team will follow applicable conventions from this guideline, including:

- Using PascalCase for type names and enum values.
- Using camelCase for functions, properties, and local variables.
- Avoiding the use of the I prefix for interface names.
- Separating logical components into appropriate files.
- Exporting types and functions only when they are required by other modules.

- Placing type definitions before their use where practical.
- Preferring undefined over null.
- Using double quotation marks for strings.
- Using four spaces for indentation.

Some standards in the Microsoft TypeScript Coding Guidelines do not apply directly to FitFolio. The team will adopt the conventions that are relevant to the project instead of following every specific recommendation.

For Python code, the team will follow PEP 8 – Style Guide for Python Code, which includes conventions for indentation, naming, whitespace, imports, and the general code layout.

Coding standards will be enforced through automated formatting and linting checks, as described in Section 10.8. These checks are included in the project's CI workflow and must pass before merging. Pull request reviewers will also verify that code follows the team's coding standards before approving a merge.

13 Appendix — Generative AI Use Disclosure

This section summarizes how the team used AI while preparing the Problem Statement and Goals document and the Development Plan. It identifies the tools used, the sections affected, and how the team checked the AI-assisted material.

13.1 AI Tools Used

The following AI tools and models were used while preparing this document:

- ChatGPT (OpenAI): GPT-5.6 and GPT-5.6 Luna.
- Claude (Anthropic): Claude Opus 5.5.

13.2 How and Where Used

ChatGPT was used to help with LaTeX formatting, resolve formatting problems, identify grammar errors, and provide feedback to improve the clarity and presentation of the writing. These uses affected Section 1.3 (Stakeholders) in the Problem Statement and Goals document; Sections 7 (Workflow Plan), 8 (Project Decomposition and Scheduling), 9 (Proof of Concept Demonstration Plan), and 11 (Use of AI and LLMs) in the Development Plan; and the Team Building and Decision-Making parts of the Team Charter.

ChatGPT also provided help with LaTeX formatting and syntax in Section 3 (Stretch Goals) of the Problem Statement and Goals document. In the Development Plan, this assistance covered Sections 6 (Team Member Roles) and 12 (Coding Standard), as well as the Attendance and External Goals parts of the Team Charter.

Claude helped with table placement under "Triggers" in the Team Charter's "Stay on Track" subsection, within "Accountability and Teamwork" in the Development Plan. It provided LaTeX formatting assistance to place the table directly below the relevant text.

Generative AI assistance was limited to LaTeX formatting and syntax, grammar checks, and feedback. It was not used to generate research, references, technical content, or substantive ideas for either the Problem Statement and Goals document or the Development Plan.

13.3 Verification of AI-Generated Content

The team members manually reviewed all changes made with AI assistance before including them in the Problem Statement and Goals document and the Development Plan. For grammar and clarity suggestions, we compared the revised wording with the original text to check that it preserved our intended meaning and did not introduce new claims, requirements, or team commitments. We also checked that the wording matched the project scope and the information in related sections.

For LaTeX formatting and syntax changes, we reviewed the source changes and checked the generated PDFs. We looked for misplaced tables, text extending beyond the margins, missing content, and changes to headings or numbering. For the Triggers table in the Team Charter, we checked that it appeared below the relevant text and that its entries remained unchanged.

In cases where AI-assisted changes affected factual or technical statements, we checked them against the relevant course instructions, project information, or official documentation. Any affected references were checked against their original sources, and any affected calculations were checked independently. We also compared the completed sections with the template instructions and rubric to identify missing information. The team remains responsible for the accuracy, completeness, consistency, and appropriateness of both documents.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?

Creating the development plan before writing any code forced us to make decisions while they were still cheap to change. The clearest example came from reviewing this document itself: sections written by different members initially gave three different time limits for pull request reviews and two different definitions of an internal deadline, and one section assumed the app would run only on Android while another planned for both Android and iOS. Finding these contradictions on paper took a few minutes to fix. Finding them after development had started would have meant missed reviews, confusion about deadlines, or code written for the wrong platform.

The plan also made us identify our biggest risks before committing to the project's core design. Writing the proof of concept plan showed that FitFolio depends on two systems none of us had used before, the phones' health platforms and a market data provider with request limits, so we can test both early instead of discovering problems mid-year. Similarly, agreeing on the team charter's triggers and consequences now, while everyone is on good terms, means that if a problem does arise, we already have a fair process that no one has to invent under pressure.

At the same time, we see the plan as a starting point rather than a fixed contract. Some decisions, such as our target market data provider, may change after the proof of concept. Because every change to this document goes through a reviewed pull request, the plan can evolve while keeping a clear record of what changed and why.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

The main advantage of CI/CD is that problems are caught when a change is proposed, not after it reaches main. For our documents, this is already useful:

every pull request compiles the LaTeX files and attaches the generated PDFs, so a reviewer can see a broken table or a missing reference in the rendered document instead of discovering it on submission day. Once development begins, running the linter and test suite on every pull request means that no one's change can quietly break someone else's feature, which matters in a team of five working on separate parts of the app at the same time. Continuous deployment adds a further benefit for us: an automatic test build on every merge lets our supervisor try the latest version on his own phone, so we can get feedback throughout the year rather than only at demonstrations.

The disadvantages are real, however. First, CI/CD takes time to set up before it produces any value, and time spent configuring workflows is time not spent on features. Deployment is especially demanding for a mobile app, since building for iOS and Android requires signing keys and developer accounts that must be stored securely. Second, a failing or unreliable check blocks merging, which can slow the team down right before a deadline if a test fails for reasons unrelated to the change. Third, and most importantly, CI/CD can create a false sense of security, because it only catches what it is designed to check. Our documentation checks passed while this document still contained contradictions between sections, since compiling successfully says nothing about whether the content is consistent. For this reason, we treat CI as a complement to human review rather than a replacement for it.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Most of our disagreements did not come from arguments in meetings, but from members writing their sections separately and making different assumptions. They only became visible when we reviewed both documents together.

The first was the product's name. Our repository and team name use "Burn n' Earn", while other sections called the product FitFolio, so both names appeared across the documents. We agreed to use FitFolio as the product name everywhere, and keep Burn n' Earn as the team and repository name.

The second was which platforms to support. One section described FitFolio as an Android app, while Expected Technology planned for both Android and iOS. Since we chose React Native so that one codebase could run on both platforms, and the team has testing devices for both, we agreed to support both. To keep the proof of concept achievable, it will be demonstrated on Android first, with iOS as a stretch goal for the demonstration.

The third was which technologies to use. Our members have experience in different languages and frameworks, so each main choice had supporters of different options. Instead of deciding by preference, we wrote down one alternative for each main choice and why we did not pick it. Seeing the trade-offs side by side made the decisions much easier to agree on.

Finally, we disagreed on whether wearable devices should be part of the core project. Some members saw wearables as a natural fit for a fitness app, while others felt that supporting additional devices on top of two phone platforms

was too ambitious. We made wearables a stretch goal, so the core product can be completed first.

The main lesson was that sections written in parallel can drift apart without anyone noticing. For future deliverables, we will agree on key assumptions, such as names and platforms, before dividing the work, and review the full document together before the team deadline.

Appendix — Team Charter

This appendix sets out the expectations that team members have agreed to hold each other to throughout the project.

External Goals

The team's external goals are to use FitFolio as an opportunity to showcase the technical skills developed throughout the entirety of the Software Engineering program while producing a project that can be presented outside of the course.

The team aims to:

- Produce a high-quality project that performs well against the evaluation criteria.
- Develop a polished, usable high-quality application that is able to be demonstrated at the Capstone EXPO.
- Gain experience developing large software systems over multiple iterative development phases.
- Strengthen experience with software architecture, testing, code review, requirements, and project management.
- Create a project that team members can discuss during future technical interviews and include in portfolios.
- Gain experience working with and integrating real-world external services and managing the limitations that come with those services.

Attendance

Expectations

It is expected that team members attend scheduled meetings, arrive on time, and be prepared to discuss work assigned to them. Members should make an effort to attend all meetings with the project supervisor.

If a member expects that they will arrive late, leave early, or miss a meeting, they must notify the team as early as possible through Microsoft Teams. Any member who misses a meeting is responsible for reviewing the meeting notes and following up on any decisions, assigned tasks, or deadlines that affect their work.

Repeated attendance issues will be handled according to the thresholds defined in Table 8. Attendance falling below the team's defined 80 percent threshold or two consecutive unexcused absences will trigger the corresponding team response.

Acceptable Excuse

Acceptable reasons for missing a meeting or deadline include illness, family or personal emergencies, major academic conflicts, interviews or employment obligations, and other unexpected circumstances that prevent a member from attending or completing work.

For foreseeable conflicts, team members should provide notice in advance whenever possible. Repeated unexplained absences, or failure to complete assigned work without communicating with the team will be considered unacceptable.

In Case of Emergency

If a team member experiences an emergency that prevents them from attending a meeting or completing assigned work, they should inform the team as soon as it is reasonably possible to do so.

The team can then determine if the affected work should be reassigned, delayed, or adjusted based on the urgency of the task and the upcoming deadline. Once the team member is able to return they are responsible for reviewing decisions made during their absence and coordinating with the team about any remaining responsibilities.

Accountability and Teamwork

This subsection sets out what team members can expect from each other in terms of the quality of their work, their attitude, and staying on track.

Quality

This subsection describes the standard of preparation and work expected from every team member.

Meeting Preparation and In-Progress

The following describes how team members prepare for and document meetings:

- The meeting chair posts the agenda as a GitHub issue using an appropriate template (dependent on the meeting) at least 24 hours before the meeting. The meeting date must be visible within the issue title. Team members add their items/questions before it starts.
- Everyone arrives with a short update (done since last time, what's next, any blockers) and has their GitHub Projects cards up to date.
- Anything linked in the agenda, like a pull request or a draft section, is read before the meeting.
- All items discussed and their outcomes are documented in the meeting's GitHub issue. Once the meeting is complete, the issue shall be marked as done.

Quality Assurance

The following steps ensure each deliverable meets the course checklist and rubric before it is submitted:

- Before starting any deliverable, every team member responsible for the feature must read through the checklist and rubric. Any questions and/or doubts must be raised as soon as possible.
- Each deliverable is assigned a QA reviewer, who must not be the main author. This role rotates between team members for each deliverable.
- During the three days between the team deadline and the course deadline (see Section 8.3), the QA reviewer will go through the checklist and rubric to ensure all items are at least being met.
- Should an issue with the checklist and/or rubric arise, they must communicate through the GitHub issues and/or Teams channel for extra help from other team members.
- Team members may also request periodic check-ins with another team member who mutually agrees.

Definition of Done

Every feature that is considered to have reached a certain goal/milestone needs the following within a pull request:

- Linked to a specific GitHub issue.
- Passes all CI tests and formatting checks.
- (Unless otherwise stated) Tests for the new features with brief justification on why they exist and that they pass.
- Approved by at least one other team member who isn't the main author. In the case where all team members worked on a feature, everyone must approve it.

Attitude

This subsection describes how team members are expected to treat each other and work together.

Code of Conduct

The following sets the standard of behaviour expected from every team member:

- The team adopts the Contributor Covenant v2.1 as its code of conduct, linked from `CodeOfConduct.md` in the repository.
- All team members are expected to treat each other, the supervisor, TAs and instructors with respect, regardless of experience level or background.

Ideas and Feedback

The following describes how ideas and feedback are shared within the team:

- Every idea is welcome and will be heard out before it is judged. No idea is dismissed without a stated reason.
- Feedback critiques the work, *never* the person. Review comments should be specific and explain *why* a change is suggested.
- Disagreements should be raised openly during meetings or on the relevant issue/pull request.
- Once the team makes a decision (see Decision Making), every team member must support it.

Conflict Resolution

The following describes the steps for how conflicts between team members shall be handled:

1. Members involved first try to resolve the conflict directly and privately within 48 hours.
2. If unresolved, the issue is brought to the next team meeting, where a team member not involved acts as mediator.
3. The mediator and the agreed outcome (*not* the details of the argument) are recorded in the meeting minutes.
4. If still unresolved, or in the case of a Code of Conduct violation, the team escalates to the TA for investigation.
5. If all else fails and with agreement from the TA, the course instructor shall be notified.

Stay on Track

This subsection describes how the team tracks progress, detects when a team member is falling behind, and responds.

Tracking Progress

The following describes how progress is tracked throughout the project:

- The GitHub Projects board is the single source of truth for who is working on what. Every task must have a specific issue, an assignee and a target date.
- At each team meeting, the chair reviews the board. While blockers are the most important agenda item, any item that has been in progress for over a week without an update is discussed.
- Anything progress related discussed within the team meeting is noted within the specific meeting issue on GitHub.

- *All* deliverables have a team deadline three days before the course deadline, by which all work must be merged into `main` (see Section 8.3). The remaining three days are reserved for the QA reviewer's checklist pass (see Quality).
- There may be a case in which a pull request for a feature is not ready for review before the team deadline. The pull request can be opened regardless with a note indicating it isn't ready yet with reason as to why. The corresponding reviewer must also agree to review in short notice ahead of time.
- Before each productivity report, the team reviews the metrics collected for the TA and instructor (meeting attendance, commits, pull requests, issues, etc.). These are not treated as scores but rather signals for team performance. For example, a small documentation change and a large code change both count as one commit, which is why these numbers are not treated as scores.

Triggers

Triggers are used as a means of detecting if there is a performance issue with one or more team members. The triggers in Table 8 are to be reported in each productivity report. Absences or missed deadlines with an acceptable excuse (see Attendance) do not count.

Trigger	**Threshold**	**Measured from**
Meeting attendance	Below 80% of team meetings in a deliverable period, or 2 unexcused misses in a row	Meeting issues
Missed team deadline	2 or more in a term	Issues and Pull Requests
Review turnaround	Requested review not completed within 72 hours, 2 or more times in a deliverable period	Pull request timestamps
Responsiveness	Mentions not acknowledged within the time set in the Communication Plan, 3 or more times in a deliverable period	GitHub and Teams
Contribution	No closed issues, merged PRs or reviews for 2 consecutive weeks	GitHub

Table 8: Triggers for detecting performance issues within the team

Consequences

The consequences for hitting the triggers described above are as follows:

1. **First trigger:** A teammate privately checks in with the member to find out what is going on. Work is redistributed or help is offered if needed.
2. **Second trigger:** The issue is raised at a team meeting and recorded in the minutes. The member and team agree on a written plan with specific tasks and dates.
3. **Continued triggers:** The team books a meeting with the TA, and then the course instructor if all else fails.

Recognition

Team members who hit every target over a deliverable period are recognized by getting first pick of tasks for the next deliverable.

Team Building

The team will build cohesion through regular, low-effort rituals rather than occasional large events, since consistent contact matters more than one-off activities.

- **Weekly check-in:** The first five minutes of every weekly team meeting are reserved for an informal check-in, where each member shares how they are doing, both within the project and outside it. This helps the team notice early when a member is overloaded, and adjust workloads before deadlines are affected.
- **Deliverable retrospectives:** After each deliverable is submitted, the team holds a 20-minute retrospective during the next weekly meeting. Each member shares one thing that went well and one thing to improve, and the team agrees on at least one change to try for the next deliverable. This gives every member an equal voice, keeps small frustrations from building up, and feeds directly into each deliverable's reflection.
- **Using our own product:** Once the proof of concept is working, the team will hold a weekly step challenge using FitFolio itself. This builds friendly competition within the team, and gives every member real experience as a user of the product, which will help identify usability issues early.

Decision Making

The team will aim to make decisions by consensus, since decisions that every member agrees with are more likely to be followed through. Decisions are made during the weekly team meeting, or on Microsoft Teams for decisions that cannot wait.

- **Voting:** If consensus is not reached after 15 minutes of discussion, the decision is made by majority vote. A vote requires at least four of the five members to be present or to respond on Microsoft Teams, and passes with at least three votes in favour, so that no decision is made by a minority of the team.

- **Area decisions:** Technical decisions within a specific area of the app (e.g., the health data integration or the market engine) are made by the members working on that area, who inform the rest of the team. Any member may ask for such a decision to be brought to a team vote.
- **Scope decisions:** Decisions that change the product's scope or vision, such as removing a feature, are brought to the supervisor before being adopted.
- **Recording decisions:** Every decision and its reasoning is recorded in the relevant meeting issue, so that settled decisions are not revisited without new information.

When disagreements arise, each side will explain their position with a focus on what is best for the project rather than personal preference. If the same disagreement remains unresolved after two consecutive team meetings, it will be raised with the supervisor, who will help the team reach a decision.